AGGREGATE FUNCTIONS

Aggregate Functions

Aggregate functions perform calculations on multiple rows and return a single value.

STEP 1: Create Table

```
CREATE TABLE Employee (
 id INT,
 name VARCHAR(100),
 department VARCHAR(30),
 salary DECIMAL(10,2)
 );
```

STEP 2: Insert Records

```
INSERT INTO Employee VALUES
 (1,'Rahul','IT',50000),
 (2,'Anjali','HR',35000),
 (3,'Kiran','Finance',45000),
 (4,'Bhanu','IT',60000),
 (5,'Ramesh','Sales',40000),
 (6,'Priya','HR',38000),
 (7,'Arjun','Finance',52000),
 (8,'Mounika','Marketing',47000),
 (9,'Sunil','IT',55000),
 (10,'Keerthi','Sales',42000);
```

SELECT Queries

Q1. Find the total salary of all employees.

```
SELECT SUM(salary) AS Total_Salary
 FROM Employee;
```

Q2. Find the average salary.

```
SELECT AVG(salary) AS Average_Salary
 FROM Employee;
```

Q3. Find the highest salary.

```
SELECT MAX(salary) AS Highest_Salary
 FROM Employee;
```

Q4. Find the lowest salary.

```
SELECT MIN(salary) AS Lowest_Salary
 FROM Employee;
```

Q5. Find the total number of employees.

```
SELECT COUNT(*) AS Total_Employees
 FROM Employee;
```

Q6. Find the number of employees in the IT department.

```
SELECT COUNT(*) AS IT_Employees
 FROM Employee
 WHERE department='IT';
```

Q7. Find the total salary of IT employees.

```
SELECT SUM(salary)
 FROM Employee
 WHERE department='IT';
```

Q8. Find the average salary of HR employees.

```
SELECT AVG(salary)
 FROM Employee
 WHERE department='HR';
```

Q9. Find the maximum salary in the Finance department.

```
SELECT MAX(salary)
 FROM Employee
 WHERE department='Finance';
```

Q10. Find the minimum salary in the Sales department.

```
SELECT MIN(salary)
 FROM Employee
 WHERE department='Sales';
```

Q11. Count employees whose salary is greater than 45000.

```
SELECT COUNT(*)
 FROM Employee
 WHERE salary>45000;
```

Q12. Find the total salary of HR and IT employees.

```
SELECT SUM(salary)
 FROM Employee
 WHERE department IN ('HR','IT');
```

Difference Between COUNT(*) and COUNT(column_name) in MySQL

Feature	**COUNT(*)**	**COUNT(column_name)**
Counts	All rows in the table	Only rows where the specified column is **NOT NULL**
Includes NULL values	Yes (counts every row)	No (ignores NULL values)
Arguments	*	Column name
Purpose	Count total number of records	Count non-NULL values in a specific column

Clauses in MYSQL

In MySQL, **clauses** are used to control **how data is selected, filtered, grouped, sorted, or modified** in SQL queries.
They make SQL statements more meaningful and powerful.

Think of clauses like **instructions** given to the database.

where clause

==========

filter records based on specified conditions where clauses can be used as comparison operators, logical operators and ..etc operators.

syntax

----

select column1,column2....

from table

where condition.

Group by clause

==========

The group by clause is used to group rows having the same values in one or more columns. It is commonly used with aggregate functions such as sum(),min(),max(),avg(),count().

syntax

----

select column-name,aggregate_function(column-name)

from table-name

group by column_name.

Having Clause

The having clause is used to filter grouped records after group by clause. it is similar to where but it works on grouped instead of individual rows.

syntax

----

select column-name,

aggregate_function(column-name)

from table-name

group by column-name

having condition.

Order By Clause

----

the order by clause is used to sort the result set in ascending(ASC)or descending(Desc)order.

asc-->Ascending order by default.

Desc-->Descending order

syntax:

----

select column-name

from table-name

order by column-name asc;

Example

—--

CREATE TABLE STUDENTS (

StudentID INT AUTO_INCREMENT PRIMARY KEY,

```
    Name VARCHAR(50),
    Age INT,
    Department VARCHAR(30),
    Marks INT,
    City VARCHAR(30)
);

INSERT INTO STUDENTS (Name, Age, Department, Marks, City) VALUES
('Anil', 20, 'CSE', 85, 'Hyderabad'),
('Sneha', 22, 'ECE', 75, 'Mumbai'),
('Ravi', 20, 'CSE', NULL, 'Hyderabad'),
('Divya', 23, 'MECH', 65, NULL),
('Kiran', 22, 'ECE', 75, 'Pune'),
('Varun', 21, 'CSE', 92, 'Chennai'),
('Priya', 20, 'EEE', 88, 'Delhi'),
('John', 22, 'MECH', 55, 'Mumbai'),
('Rekha', 21, 'ECE', 75, 'Pune'),
('Ramesh', 23, 'EEE', NULL, 'Delhi');

select*from students;
```

WHERE Clause Exercises

Question 1

Display all students who belong to the CSE department.

Query

```
SELECT *
 FROM students
 WHERE Department = 'CSE';
```

Question 2

Display all students whose marks are greater than 80.

Query

```
SELECT *
 FROM students
 WHERE Marks > 80;
```

Question 3

Display all students who are 22 years old.

Query

```
SELECT *
 FROM students
 WHERE Age = 22;
```

Question 4

Display all students who belong to Mumbai.

Query

```
SELECT *
 FROM students
 WHERE City = 'Mumbai';
```

Question 5

Display all students whose marks are less than 70.

Query

```
SELECT *
 FROM students
 WHERE Marks < 70;
```

Question 6

Display all students whose marks are between 70 and 90.

Query

```
SELECT *
 FROM students
 WHERE Marks BETWEEN 70 AND 90;
```

Question 7

Display all students whose age is not equal to 20.

Query

```
SELECT *
 FROM students
 WHERE Age <> 20;
```

Question 8

Display all students who belong to CSE and have marks greater than 80.

Query

```
SELECT *
 FROM students
 WHERE Department = 'CSE'
 AND Marks > 80;
```

Question 9

Display all students who belong to ECE or EEE department.

Query

```
SELECT *
 FROM students
 WHERE Department = 'ECE'
 OR Department = 'EEE';
```

Question 10

Display all students whose city is not Delhi.

Query

```
SELECT *
 FROM students
 WHERE City <> 'Delhi';
```

GROUP BY Clause

Question 1

Display the total number of students in each department.

Query

```
SELECT Department, COUNT(*) AS Total_Students
 FROM students
 GROUP BY Department;
```

Question 2

Display the average marks of students in each department.

Query

```
SELECT Department, AVG(Marks) AS Average_Marks
 FROM students
 GROUP BY Department;
```

Question 3

Display the highest marks scored in each department.

Query

```
SELECT Department, MAX(Marks) AS Highest_Marks
 FROM students
 GROUP BY Department;
```

Question 4

Display the lowest marks scored in each department.

Query

```
SELECT Department, MIN(Marks) AS Lowest_Marks
 FROM students
 GROUP BY Department;
```

Question 5

Display the total marks obtained by students in each department.

Query

```
SELECT Department, SUM(Marks) AS Total_Marks
 FROM students
 GROUP BY Department;
```

Question 6

Display the number of students in each city.

Query

```
SELECT City, COUNT(*) AS Total_Students
 FROM students
 GROUP BY City;
```

Question 7

Display the average age of students in each department.

Query

```
SELECT Department, AVG(Age) AS Average_Age
 FROM students
 GROUP BY Department;
```

Question 8

Display the total marks obtained by students in each city.

Query

```
SELECT City, SUM(Marks) AS Total_Marks
 FROM students
 GROUP BY City;
```

Question 9

Display the highest marks in each city.

Query

```
SELECT City, MAX(Marks) AS Highest_Marks
 FROM students
 GROUP BY City;
```

Question 10

Display the total number of students according to age.

Query

```
SELECT Age, COUNT(*) AS Total_Students
 FROM students
 GROUP BY Age;
```

HAVING Clause

Question 1

Display departments having more than 2 students.

Query

```
SELECT Department, COUNT() AS Total_Students
 FROM students
 GROUP BY Department
 HAVING COUNT() > 2;
```

Question 2

Display departments whose average marks are greater than 75.

Query

```
SELECT Department, AVG(Marks) AS Average_Marks
 FROM students
 GROUP BY Department
 HAVING AVG(Marks) > 75;
```

Question 3

Display cities having more than one student.

Query

```
SELECT City, COUNT() AS Total_Students
 FROM students
 GROUP BY City
 HAVING COUNT() > 1;
```

Question 4

Display departments whose total marks are greater than 150.

Query

```
SELECT Department, SUM(Marks) AS Total_Marks
 FROM students
 GROUP BY Department
 HAVING SUM(Marks) > 150;
```

Question 5

Display age groups having more than two students.

Query

```
SELECT Age, COUNT() AS Total_Students
 FROM students
 GROUP BY Age
 HAVING COUNT() > 2;
```

Question 6

Display cities whose average marks are greater than 70.

Query

```
SELECT City, AVG(Marks) AS Average_Marks
 FROM students
 GROUP BY City
 HAVING AVG(Marks) > 70;
```

Question 7

Display departments whose highest marks are greater than 90.

Query

```
SELECT Department, MAX(Marks) AS Highest_Marks
 FROM students
 GROUP BY Department
 HAVING MAX(Marks) > 90;
```

Question 8

Display departments whose lowest marks are less than 60.

Query

```
SELECT Department, MIN(Marks) AS Lowest_Marks
 FROM students
 GROUP BY Department
 HAVING MIN(Marks) < 60;
```

Question 9

Display cities whose total marks are greater than 100.

Query

```
SELECT City, SUM(Marks) AS Total_Marks
 FROM students
 GROUP BY City
 HAVING SUM(Marks) > 100;
```

Question 10

Display departments having exactly 2 students.

Query

```
SELECT Department, COUNT() AS Total_Students
 FROM students
 GROUP BY Department
 HAVING COUNT() = 2;
```

ORDER BY Clause

Question 1

Display all students ordered by name.

Query

```
SELECT *
 FROM students
 ORDER BY Name;
```

Question 2

Display all students ordered by marks in descending order.

Query

```
SELECT *
 FROM students
 ORDER BY Marks DESC;
```

Question 3

Display all students ordered by age.

Query

```
SELECT *
 FROM students
 ORDER BY Age;
```

Question 4

Display all students ordered by department.

Query

```
SELECT *
 FROM students
 ORDER BY Department;
```

Question 5

Display all students ordered by city.

Query

```
SELECT *
 FROM students
 ORDER BY City;
```

Question 6

Display all students ordered by department and marks in descending order.

Query

```
SELECT *
 FROM students
 ORDER BY Department, Marks DESC;
```

Question 7

Display all students ordered by age in descending order.

Query

```
SELECT *
 FROM students
 ORDER BY Age DESC;
```

Question 8

Display all students ordered by city and name.

Query

```
SELECT *
 FROM students
 ORDER BY City, Name;
```

Question 9

Display all students ordered by marks and age.

Query

```
SELECT *
 FROM students
 ORDER BY Marks DESC, Age ASC;
```

Question 10

Display all students ordered by department, city, and name.

Query

```
SELECT *
 FROM students
 ORDER BY Department, City, Name;
```

WHERE + GROUP BY + HAVING + ORDER BY (Order of Execution)

SQL Execution Order

1. **FROM**
2. **WHERE**
3. **GROUP BY**
4. **HAVING**
5. **SELECT**
6. **ORDER BY**

Question 1

Display the average marks of each department considering only students older than 20 years. Show departments whose average marks are greater than 70 and sort the results by average marks in descending order.

Query

```
SELECT Department,
 AVG(Marks) AS Average_Marks
 FROM students
 WHERE Age > 20
 GROUP BY Department
 HAVING AVG(Marks) > 70
 ORDER BY Average_Marks DESC;
```

Question 2

Display the total number of students in each city considering only students whose marks are greater than 70. Show cities having more than one student and sort by total students.

Query

```
SELECT City,
 COUNT() AS Total_Students
 FROM students
 WHERE Marks > 70
 GROUP BY City
 HAVING COUNT() > 1
 ORDER BY Total_Students DESC;
```

Question 3

Display the total marks of each department considering only students from Hyderabad and Pune. Show departments whose total marks are greater than 100 and order by total marks.

Query

```
SELECT Department,
 SUM(Marks) AS Total_Marks
 FROM students
 WHERE City IN ('Hyderabad','Pune')
 GROUP BY Department
 HAVING SUM(Marks) > 100
 ORDER BY Total_Marks DESC;
```

Question 4

Display the highest marks obtained in each city considering only students from the ECE department. Show cities whose highest marks are greater than 70 and order by highest marks.

Query

```
SELECT City,
 MAX(Marks) AS Highest_Marks
 FROM students
```

```
WHERE Department='ECE'
GROUP BY City
HAVING MAX(Marks) > 70
ORDER BY Highest_Marks DESC;
```

Question 5

Display the average age of students in each department considering only students whose city is not NULL. Show departments whose average age is greater than 20 and sort alphabetically.

Query

```
SELECT Department,
 AVG(Age) AS Average_Age
 FROM students
 WHERE City IS NOT NULL
 GROUP BY Department
 HAVING AVG(Age) > 20
 ORDER BY Department;
```